

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

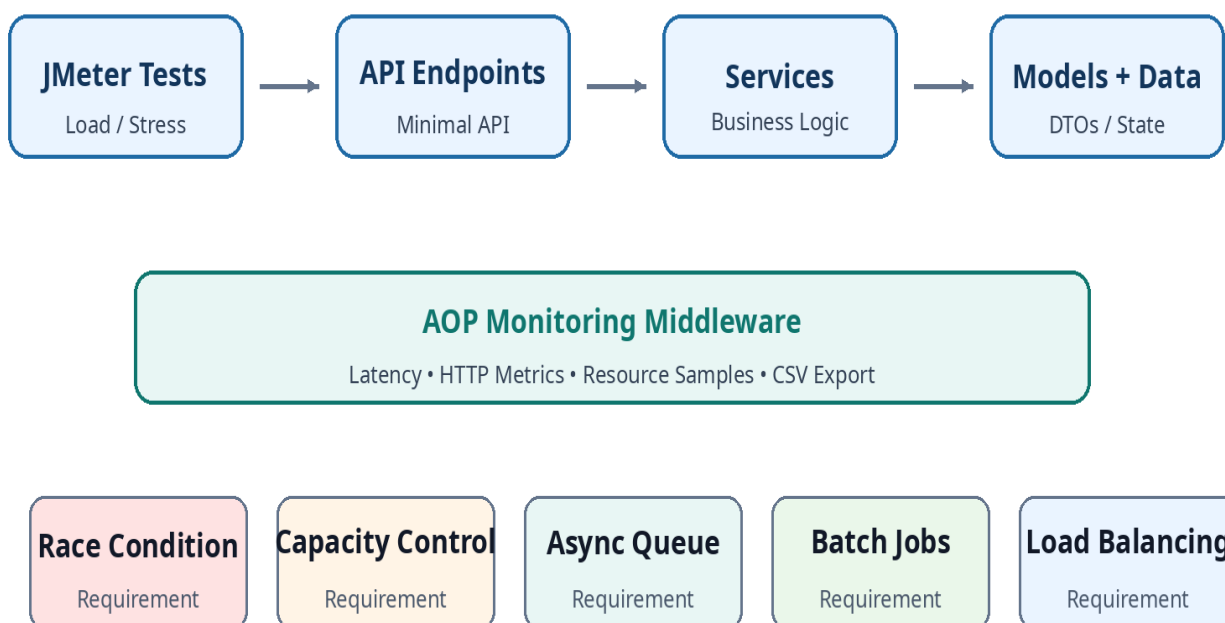
(High-Performance E-Commerce Backend Engine)

أسماء الطلاب:

- أنس جهاد بكور
- زينة غيث خليل
- شام فادي باكير
- سارة فادي صالحة

- ملخص تنفيذي ومؤشرات الإنجاز
- بنية المشروع وطبقة AOP للمراقبة
- شرح المتطلبات الخمسة: المشكلة، الحلول المقترحة، الحل المختار، ونتائج الاختبار
- مخططات توضيحية مرتبطة بالمشروع
- ملف JMeter وصور نتائج التنفيذ
- قابلية البنية لاستكمال بقية المتطلبات

Parallel E-Commerce Backend Architecture



1. ملخص تنفيذي

يهدف المشروع إلى بناء محرك Backend لمنظومة تجارة إلكترونية قادرة على التعامل مع طلبات متزامنة مع الحفاظ على سلامة البيانات، ضبط الموارد، وتحسين زمن الاستجابة. في هذه المرحلة تم التركيز على أول خمسة متطلبات لأنها تمثل أساس البنية التي سنبني عليها المتطلبات اللاحقة.

المتطلب	طريقة الحل والاثبات
حماية البيانات من Race Condition	قبل/بعد JMeter +
إدارة الموارد والتحكم بالسعة	SemaphoreSlim + Metrics
المعالجة غير المتزامنة	Queue + Background Worker
Batch Processing	Background Job + Chunks
Load Balancing	Round Robin + Health Check

2. بنية المشروع وطبقة المراقبة

تم تقسيم المشروع إلى طبقات واضحة لتجنب خلط منطق الاختبار مع منطق المعالجة

الطبقة / المجلد	الدور داخل المشروع
Endpoints	تعريف مسارات API التي يستخدمها Swagger و JMeter
Services	تنفيذ منطق التزامن Queue، Batch، Load Balancing،
Models	نماذج الاستجابة والـ DTOs والـ Snapshots الخاصة بالمقاييس
Middleware	تطبيق AOP لمراقبة زمن التنفيذ والطلبات النشطة
Docs	التوثيق، التقرير، وصور الإثبات
Tests/JMeter	ملفات JMX الرسمية ونتائج الاختبار

تطبيق AOP للمراقبة:

تمت إضافة طبقة PerformanceMonitoringMiddleware بحيث تلتف حول كل HTTP Request وتجمع زمن التنفيذ، حالة الطلب، وعدد الطلبات النشطة بدون تكرار كود القياس داخل كل Endpoint

```
Middleware/PerformanceMonitoringMiddleware.cs
Services/PerformanceMetricsService.cs
Services/ResourceMonitoringService.cs
GET /monitoring/metrics
GET /monitoring/resources
POST /monitoring/reset
```

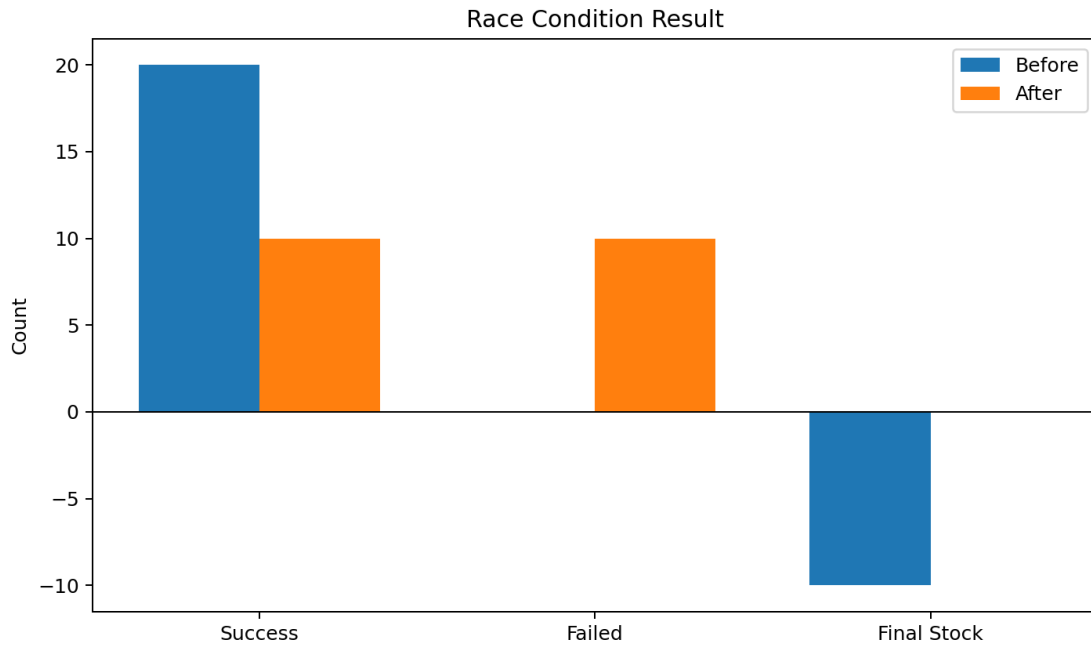
3. حماية البيانات المشتركة من التضارب | Requirement 01

المشكلة الأساسية هي أن عدة مستخدمين قد يحاولون شراء نفس المنتج في نفس اللحظة. إذا كان المخزون 10 قطع فقط ووصل 20 طلب شراء متزامن بدون حماية، فقد تنجح طلبات أكثر من الكمية المتوفرة بسبب Race Condition

مميزات وسوء كل حل	مجموعة حلول مقترحة
سريع ظاهرياً لكنه يسبب فساداً في المخزون	بدون حماية
مناسب للمحاكاة الحالية ويمنع أكثر من Thread من تعديل المخزون في نفس اللحظة	lock محلي
أنسب للإنتاج عند استخدام قاعدة بيانات حقيقي	Transaction / Row Lock
مناسب لاحقاً للأنظمة كثيرة القراءة وقليلة التصادم	Optimistic Locking

الحل المختار في هذه المرحلة هو lock محلي لإثبات حماية القسم الحرج Critical Section النتيجة قبل الحل كانت نجاح 20 عملية رغم توفر 10 قطع فقط، أما بعد الحل فنجحت 10 عمليات وفشلت 10 عمليات وأصبح المخزون النهائي 0

```
POST /demo/race/before
POST /demo/race/after
```



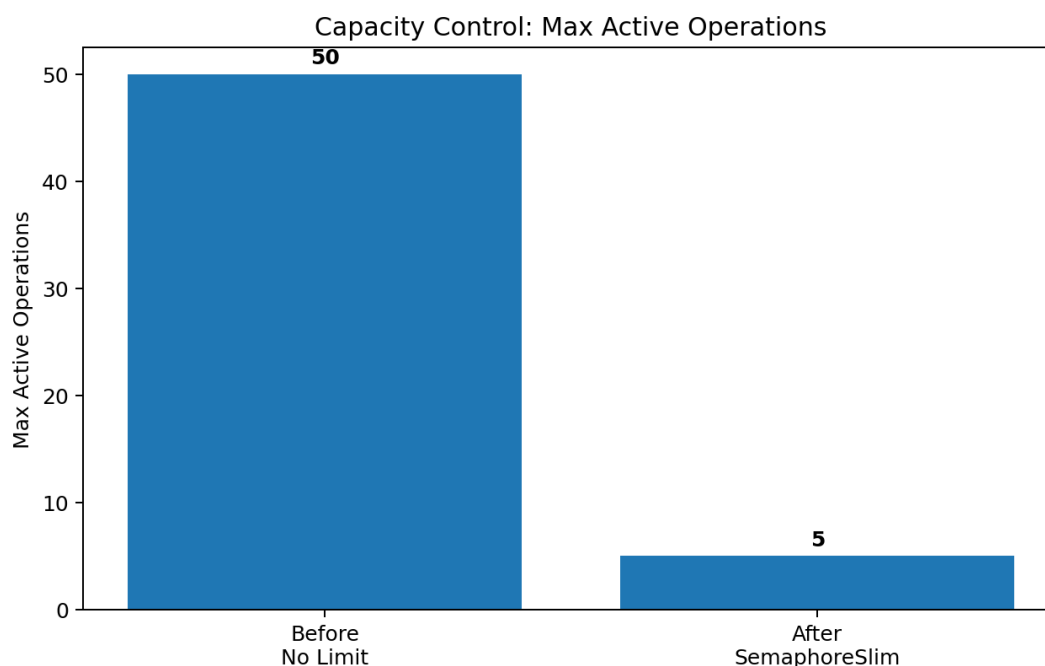
4. إدارة الموارد والتحكم بالسعة | Requirement 02

الهدف هو منع النظام من تشغيل عدد غير محدود من العمليات الثقيلة في نفس اللحظة قبل الحل كانت كل العمليات تعمل دفعة واحدة، أما بعد الحل تم تحديد عدد العمليات المتوازية باستخدام SemaphoreSlim

مميزات وسرعة كل حل	مجموعة الحلول المقترحة
قد يكون سريعاً مع الحمل الصغير لكنه خطر عند الضغط العالي	بدون حد للتوازي
يعطي حداً واضحاً وقابلاً للقياس للعمليات النشطة	SemaphoreSlim
مناسب للمهام الخلفية لكنه ليس دائماً مناسباً لعمليات تحتاج جواباً مباشراً	Queue + Workers

تم اختيار SemaphoreSlim بقيمة `MaxParallelOperations = 5` في الاختبار وصل عدد العمليات النشطة قبل الحل إلى 50 ، وبعد الحل بقي الحد الأعلى 5 فقط هذا يثبت أن البنية لا تسمح باستهلاك موارد غير مضبوط

```
POST /capacity/reset
POST /capacity/before/work
POST /capacity/after/work
GET /capacity/metrics
```



طبعا:

قد يزيد زمن الاستجابة بعد الحل لأن الطلبات الزائدة تنتظر دورها، لكن هذا التأخير مقبول لأنه يحافظ على استقرار النظام ويمنع الاستهلاك المفرط للموارد

5. المعالجة غير المتزامنة باستخدام Queue | Requirement 03

في التجارة الإلكترونية توجد مهام لا يحتاج المستخدم إلى انتظارها مثل إرسال الإشعار أو تجهيز الفاتورة نقل هذه المهام إلى Queue يقلل زمن استجابة الطلب الأساسي ويحافظ على قابلية النظام للتوسع

مجموعة الحلول المقترحة	مميزات وسوء كل حل
Synchronous Processing	بسيط لكنه يجعل المستخدم ينتظر مهاماً ثانوية
Channel + BackgroundService	حل مناسب للمشروع ويثبت المفهوم بوضوح.

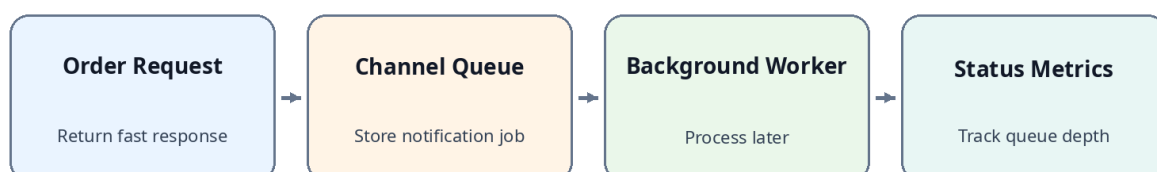
تم اختيار <Channel<NotificationJob> مع BackgroundService الطلب /orders/after-async يعيد الرد بسرعة ثم يعالج الإشعار في الخلفية مثلاً حسب نتائج JMeter ، انخفض متوسط زمن الطلب المتزامن من حوالي 3353ms إلى حوالي 323ms في المسار غير المتزامن

```

POST /notifications/reset
POST /orders/before-sync
POST /orders/after-async
GET /notifications/status

```

Async Queue Flow



Designed for load testing, monitoring, and staged delivery evidence.

6. معالجة البيانات الضخمة على دفعات | Requirement 04

معالجة جرد المبيعات اليومية كسجل تلو الآخر قد تصبح بطيئة جداً عند زيادة عدد السجلات لذلك تم اعتماد Background Job يقسم البيانات إلى Chunks مع حد واضح للتوازي حتى لا تتحول المعالجة إلى استهلاك مفرط للموارد

مميزات وسوء كل حل	مجموعة الحلول المقترحة
سهل لكنه بطيء مع البيانات الكبيرة	Sequential Processing
قد يكون سريعاً لكنه خطر على الموارد	Parallel All Records
يوفر توازناً بين السرعة والاستقرار	Chunks + Bounded Parallelism

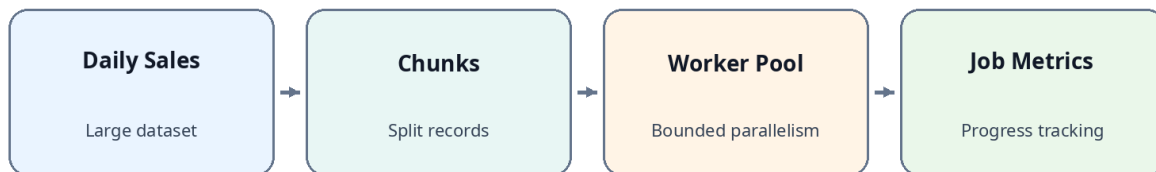
تم اختيار Background Job + Chunk Processing يبدأ endpoint الوظيفة ويعيد job_Id بسرعة، ثم تتم المعالجة في الخلفية مع إمكانية متابعة الحالة عبر /batch/jobs و /batch/metrics

```

POST /batch/reset
POST /batch/before/sequential
POST /batch/after/start-daily-sales-job
GET /batch/jobs
GET /batch/metrics

```

Batch Processing Flow



Designed for load testing, monitoring, and staged delivery evidence.

7. توزيع الأحمال | Requirement 05

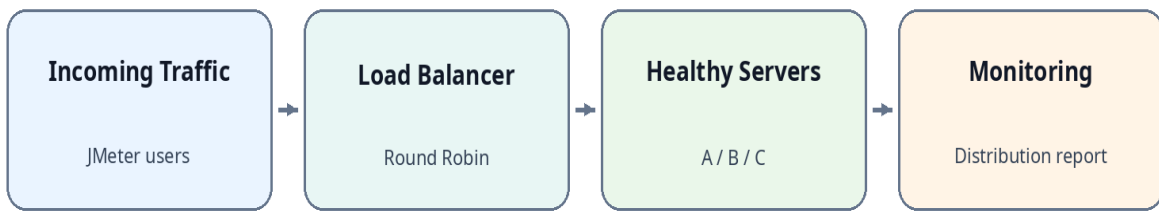
قبل الحل كانت الطلبات تذهب إلى خادم واحد فقط وهذا يجعل Server A نقطة اختناق بعد الحل تم استخدام Round Robin لتوزيع الطلبات على الخوادم الصحية، مع دعم Health Check لتجنب إرسال الطلبات إلى خادم غير صحي

مميزات وسرعة كل حل	مجموعة الحلول المقترحة
بسيط لكنه غير قابل للتوسع	Single Server
مناسب لأن الخوادم في المحاكاة متساوية	Round Robin
أدق عند اختلاف مدة الطلبات لكنه أعقد	Least Connections
مفيد عند اختلاف قدرات الخوادم	Weighted Round Robin

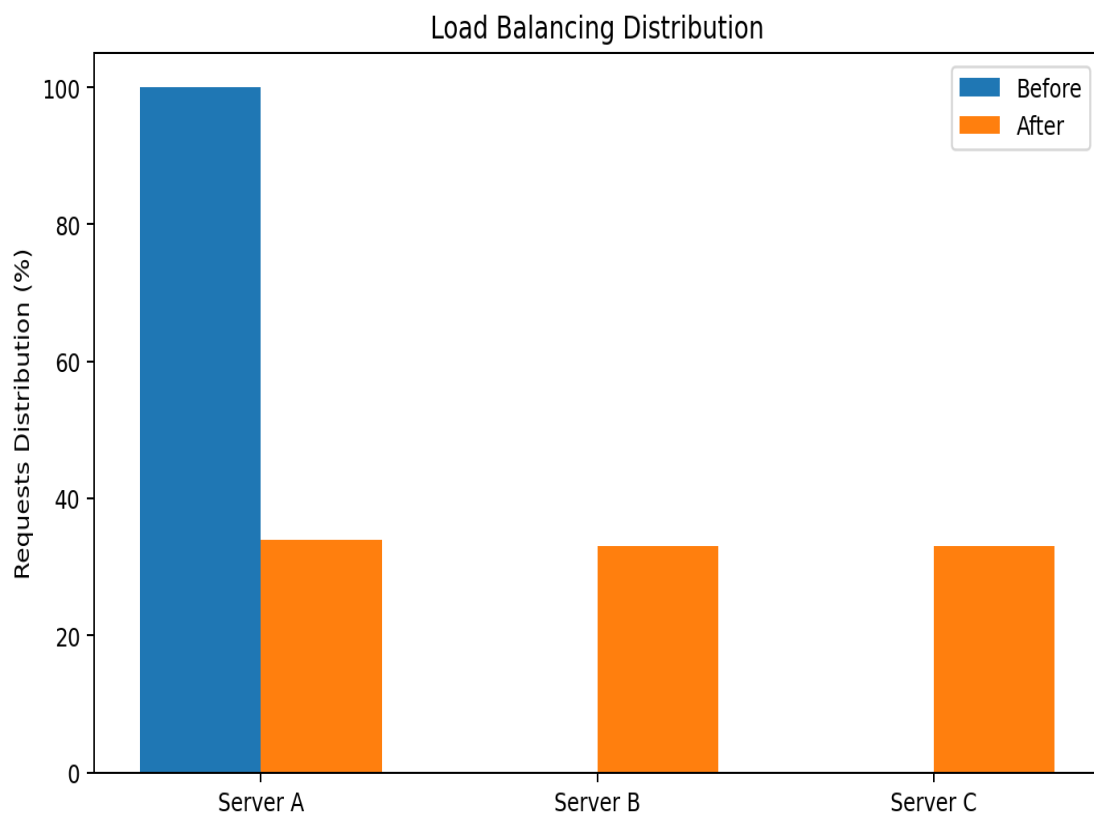
تم اختيار Round Robin لأنه يناسب محاكاة خوادم متشابهة. كما أضيف Health Check مبسط بحيث يتم استبعاد الخوادم غير الصحية من التوزيع

```
POST /load-balancer/reset
POST /load-balancer/before/request
POST /load-balancer/after/request
GET /load-balancer/metrics
POST /demo/load-balancing/after-health-check
```


Load Balancing Flow

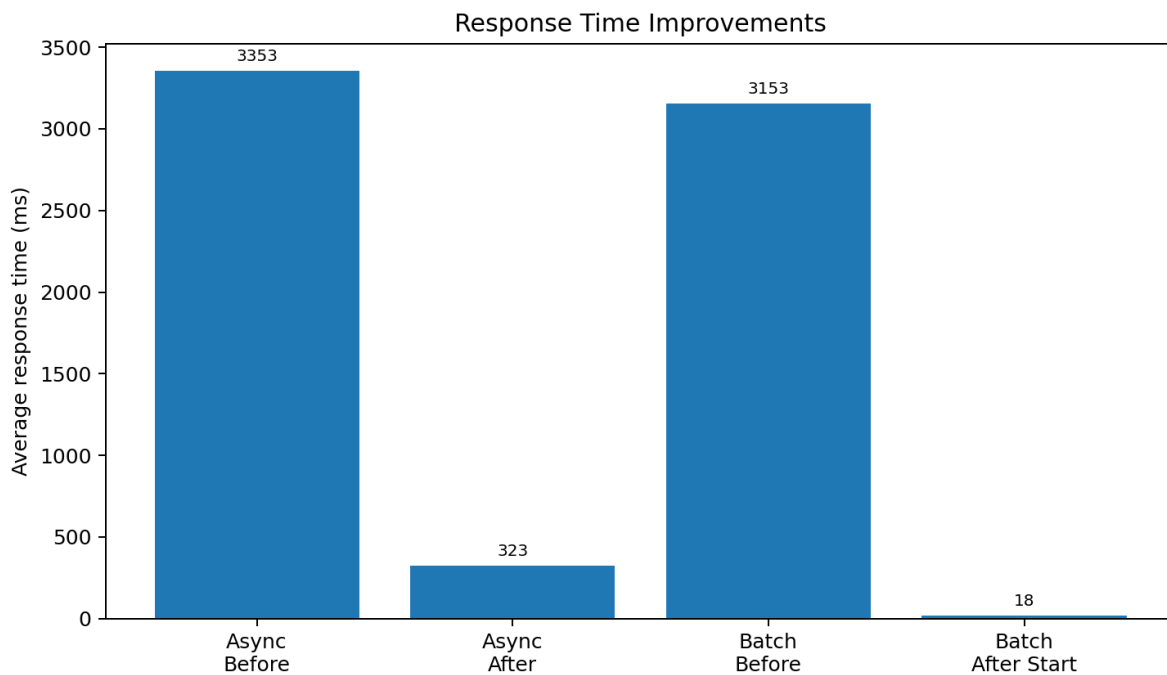


Designed for load testing, monitoring, and staged delivery evidence.



8. مقارنة أداء مختصرة

يوضح الشكل التالي أن بعض التحسينات لا تعني دائماً تقليل الزمن فقط بل تعني أيضاً تحسين الاستقرار وقابلية التحكم في Async Queue و Batch Job، الهدف الأساسي هو إخراج المهام البطيئة من مسار الاستجابة المباشرة للمستخدم



الشكل 8: مقارنة زمن الاستجابة في بعض المسارات قبل وبعد التحسين.

9. اختبار JMeter وصور التنفيذ

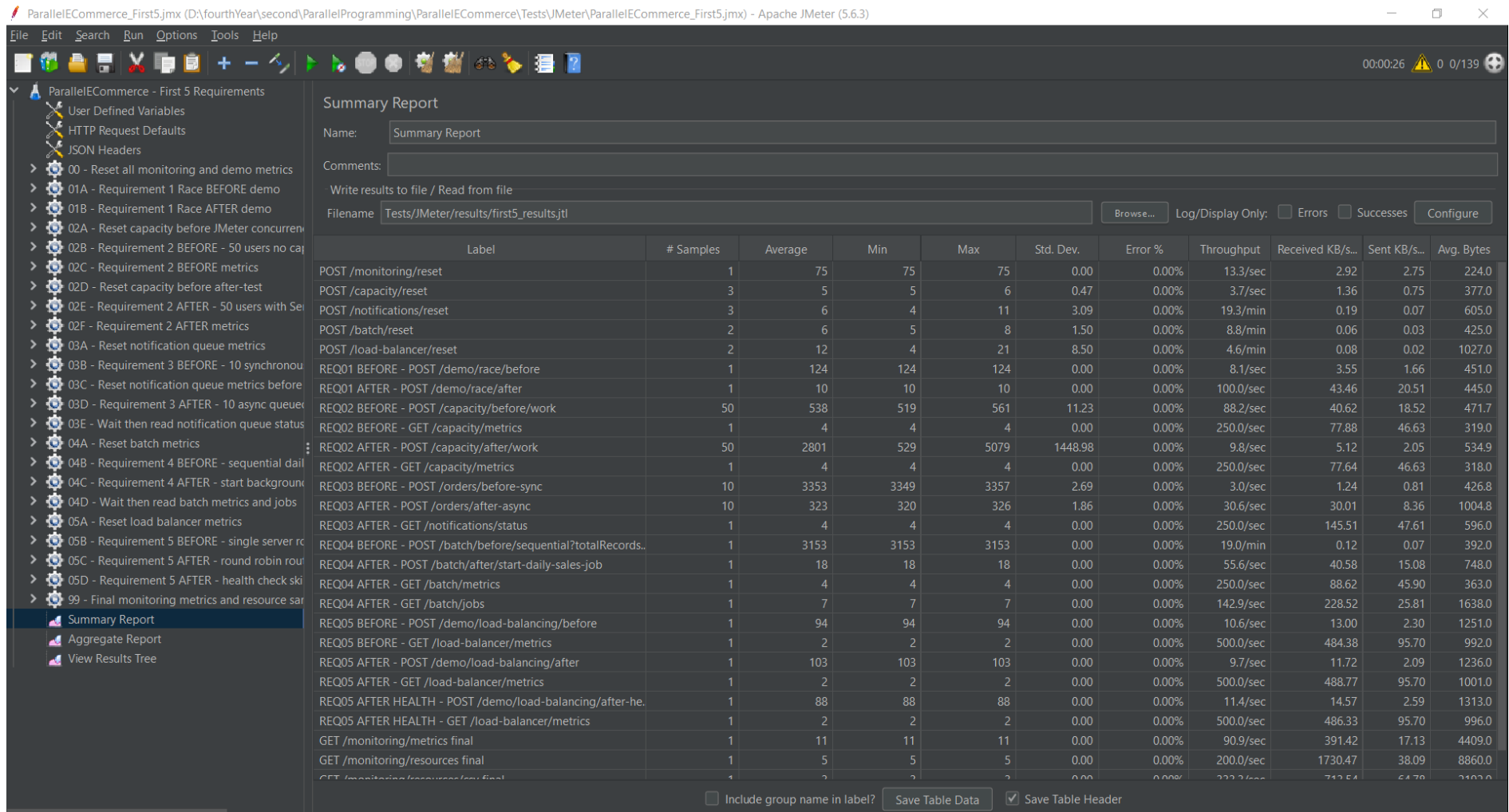
تم تجهيز ملف JMeter الرسمي داخل المشروع كما طلب الدكتور، وتم تشغيله على أول خمسة متطلبات. يغطي الملف سيناريوهات Load Balancing، Batch Processing، Async Queue، Capacity Control، Race Condition، قراءة المقاييس النهائية من Monitoring.

```
Tests/JMeter/ParallelECommerce_First5.jmx
Docs/Screenshots/jmeter_first5_summary_report.PNG
Docs/Screenshots/jmeter_first5_view_results_tree.PNG
```

نتيجة التنفيذ:

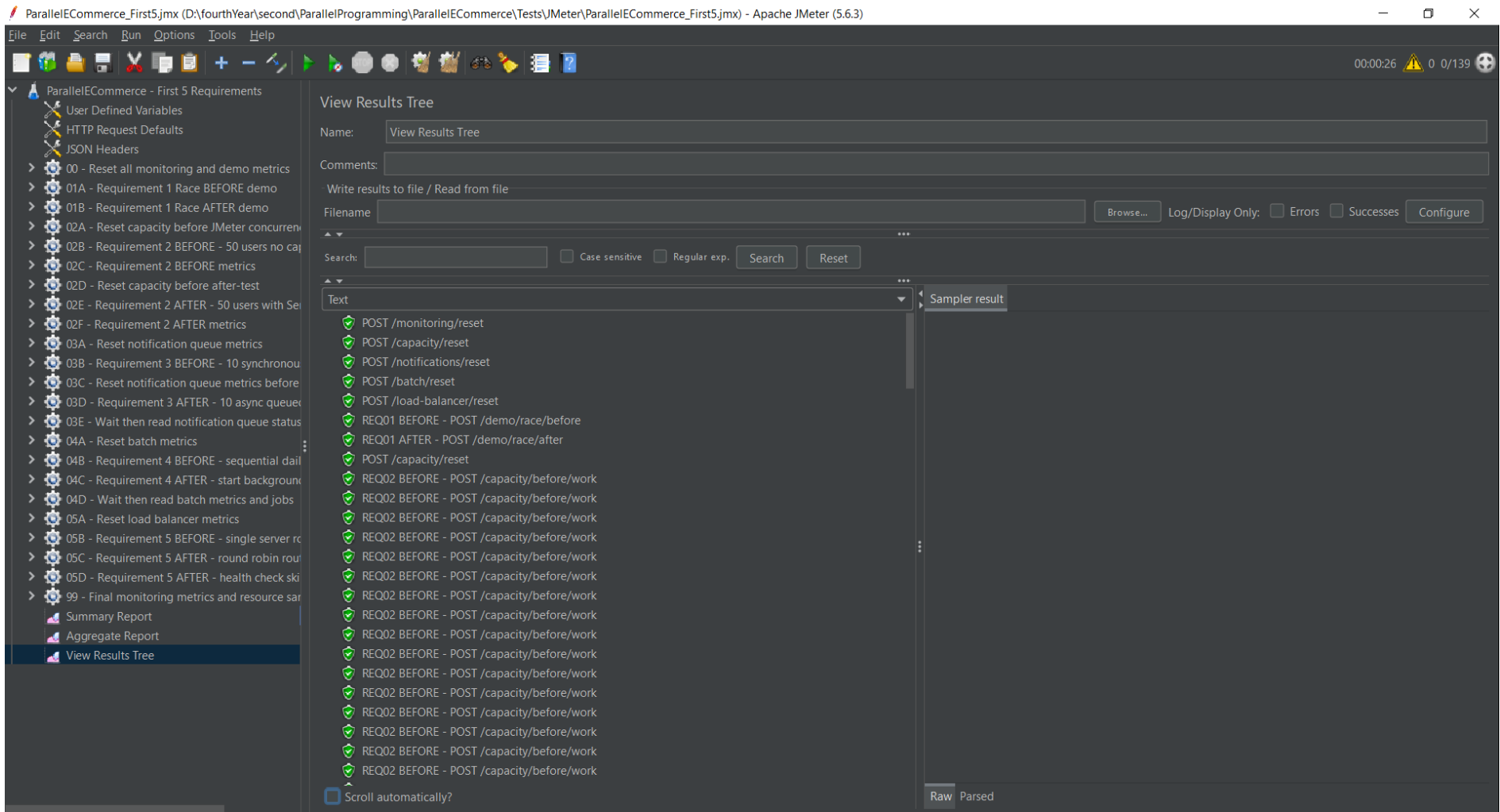
أظهرت صورة Summary Report أن Error % = 0.00% ، كما أظهر View Results Tree أن الطلبات ناجحة باللون الأخضر

صورة Summary Report من JMeter



الشكل 9: Summary Report يظهر نتائج تشغيل ملف JMX لأول خمسة متطلبات مع. Error % = 0.00%

صورة View Results Tree من JMeter



الشكل 10: View Results Tree يظهر نجاح الطلبات باللون الأخضر

